CS 61A — Week 2

August 31–September 6: Control, Higher-Order Functions, Environments
Composing Programs 1.4–1.6; Lab1, Discussion1, HW2, Hog

§1 Choose which statements run

The question. Why does a second `if` sometimes run after the first, while an `elif` does not? Why can `0 and 1/0` finish without an error?

Week 1 tools. A call evaluates its operator, then arguments left to right, then applies the function. Assignment evaluates all right-hand values before changing bindings. A user-defined function call gets a new local frame. `print` has an output effect and returns `None`; `return` ends the current call. An expression statement such as `mul(x,x)` discards its computed value. The lab's `what_prints` prints `Hello World!`, returns `'Goodbye.'`, then has an unreachable later print. At the prompt, the returned string appears with quotes; printing a string displays its text without quotes. These rules remain unchanged.

A **compound statement** consists of clauses, each with a colon-ended header and an indented suite of statements. Suites execute in order; a return redirects control immediately. Use consistent spaces for indentation. In `if`/`elif`/`else`, execute only the first true clause, or `else` if none is true. Two separate `if` statements make two separate choices.

Example (Lecture 3, slide7)

For $x = 10$, separate tests `if x > 2` and `if x > 0` print both `big` and `positive`. An `if x > 2`/`elif x > 0` chain prints only `big`. For $x = 1$ the chain prints `less big`; for $x = -1$ its `else` prints `not pos`. A negative number is true in a Boolean context even though `x > 0` is false.

A **Boolean context** uses a value's truth to choose control. Among values seen so far, `False`, numeric zero, `''` (empty text), and `None` are false; other numbers and nonempty strings are true. Comparisons `< > <= >= == !=` produce `True`/`False`; `=` assigns instead of comparing.

Proposition (Short-circuit rules — Lectures3–4; CP1.5)

`a and b`	Evaluate `a`; if false, return that value and skip `b`; otherwise evaluate and return `b`.
`a or b`	Evaluate `a`; if true, return that value and skip `b`; otherwise evaluate and return `b`.
`not a`	Evaluate `a`; return the Boolean opposite of its truth value.

1. In `0 and 1/0`, the first operand is false, so the result is 0; division is never attempted. In `2 or 1/0`, the result is 2 for the same reason with a true first operand.
2. `2 and 7` returns 7, `0 or 7` returns 7, and `not 0` returns `True`. Only `not` must return a Boolean rather than an operand value.
3. These operators are not ordinary two-argument function calls: an ordinary call evaluates *both* argument expressions before applying its function.

Printing a function result exactly once — Lecture3, slides4–5. For a pure `f(x)` returning $x + 1$, compare the following bodies when called on 3:

Body	printed lines	return value
`return f(x)`	none	4
`print(f(x))`	4	`None`
`f(x)`	none	`None`
`return print(f(x))`	4	`None`

To print *and* return the result, store it once. Here `f` is supplied as an argument; `increment` is the pure example.

```
def increment(x):
    return x + 1

def print_once(x, f):
    y = f(x)
    print(y)
    return y
```

`print_once(2, increment)` prints 3 and returns 3. Writing `print(f(x))` followed by `return f(x)` instead calls `f` twice. If `f = print`, that repeats its output effect. Lecture3 slide5's caption labels the twice-called version C; the actual twice-called code is B.

Shortcuts

- To check a long `and`/`or` expression, mark which operand is reached next and whether its truth value is already decisive. Do not evaluate a skipped error-producing call.
- Separate the printed lines from the returned value, as in Week1. A bare expression inside a function is not automatically returned.

Traps

- `5 and 2` is 2, not `True`. "And means both true" describes its truth, not its returned value.
- An `elif c > 7` after `if c > 5` can never run: every input satisfying the later test already satisfies the first.

§2 Describe what each loop variable means

The question. How do we know that a Fibonacci loop returns the requested term, rather than the term before it? How does 858 become the factors $2, 3, 11, 13$?

Proposition (Iteration and its maintained property — CP1.5; Lecture3)

A `while` statement tests its condition before every iteration. A true condition runs the entire body, then the test repeats; a false condition continues after the loop. A **loop invariant** is a property true before every test. Prove it initially, show one iteration preserves it, then use the stopping condition to identify the result. Termination needs a separate reason.

Define $F_0 = 0$, $F_1 = 1$, and $F_{k+1} = F_k + F_{k-1}$ for $k \geq 1$.

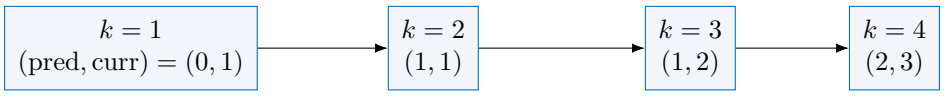
Where the proof comes from

Want. Choose enough variables to remember the progress already made.

Know. The next Fibonacci term depends on the previous two terms, not the entire earlier sequence.

Key step. Store two consecutive values and their index: before each test, `pred` is F_{k-1} and `curr` is F_k. Update both from their old values, then increase the index.

```
def fib(n):
    """Return Fibonacci number n, for n >= 1."""
    pred, curr = 0, 1
    k = 1
    while k < n:
        pred, curr = curr, pred + curr
        k = k + 1
    return curr
```



The pair changes from (F_{k-1}, F_k) to $(F_k, F_{k-1} + F_k)$. Both right-hand values must use the old pair.

1. Initially $k = 1$, so $(0, 1) = (F_0, F_1)$. One update preserves the property by the defining recurrence.
2. Each iteration increases k by 1. For integer $n \geq 1$, it reaches $k = n$, where the condition is false and `curr` is F_n.
3. Thus `fib(1)`$= 1$, `fib(3)`$= 2$, `fib(8)`$= 21$, `fib(50)`$= 12586269025$. At $n = 1$, the body never runs.

Example (Lecture3 slide9: finish the whole body)

Starting with `i, total = 0, 0`, the loop `while i < 3` executes `i = i + 1` and then `total = total + i`. The pairs after complete iterations are $(1, 1), (2, 3), (3, 6)$. Updating `i` to 3 does not skip the remaining addition in that iteration; only the *next* test stops it.

Prime factorization — Lecture3. An integer at least 2 is **prime** if its only positive divisors are 1 and itself; it is **composite** if it has a divisor strictly between them. The smallest divisor above 1 of an integer $n > 1$ is prime: a composite divisor would have a smaller divisor at least 2. Find that divisor, print it, and divide it out. (Week1 used the same search to find the largest proper partner.)

```
def smallest_factor(n):
    k = 2
    while n % k != 0:
        k = k + 1
    return k

def prime_factors(n):
    while n > 1:
        k = smallest_factor(n)
        print(k)
        n = n // k
```

remaining `n`	smallest divisor `k`	printed so far	new remaining `n`
858	2	2	429
429	3	2, 3	143
143	11	2, 3, 11	13
13	13	2, 3, 11, 13	1

Each step preserves "original input = product of printed factors × remaining `n`". The remaining positive integer strictly decreases, so it reaches 1. Factors appear in nondecreasing order: a smaller divisor of the new remainder would already have divided the old one. Source examples: $8 = 2^3$, $9 = 3^2$, $10 = 2 \cdot 5$, 11 is prime, $12 = 2^2 \cdot 3$. For input 1, print nothing.

Read decimal digits with quotient and remainder — Lab1; Discussion1. For integer $n \geq 0$ and $d > 0$, uniquely $n = dq + r$ with $0 \leq r < d$. Python gives $q = n//d$, $r = n\%d$. Thus `n % 10` is the last digit, `n // 10` removes it, and `(n // 10**k) % 10` extracts position k from the right. For 3579, division by 100 gives 35, and remainder by 10 gives digit 5.

	`/`	`//`	`%`
$1, 5$	0.2	0	1
$25, 4$	6.25	6	1
$4, 2$	2.0	2	0

All three operations raise `ZeroDivisionError` for divisor zero. Floor division with negative inputs still rounds downward: `-5 // 4` $= -2$, not -1; the decimal-digit methods here use nonnegative integers.

Example (A stopping test must detect passing, not just equality — Discussion1 Q1)

The source race loop continues while `(minutes == 0) or (tortoise - hare)`. Every nonzero difference is true, including one showing the tortoise has already passed the hare. For speeds $(2, 3)$, the hare travels 15 feet in its first five minutes and rests; at minute8 the tortoise is at 16, but the code continues and returns 15 minutes, when both distances equal 30. Test whether the tortoise is *still behind* instead. The worksheet's equality examples use "catches up" as $\geq$, despite its conflicting phrase "greater distance."

Shortcuts

- For a digit loop, write the unprocessed number and the accumulated answer separately. Removing one last digit decreases a positive integer, proving termination.
- To recognize adjacent eights, inspect `n % 100 == 88`, then remove just *one* digit so that overlapping pairs are checked.
- A factor-count loop includes n itself; a proper-factor sum stops before n. The different bounds reflect the different tasks, rather than a different loop rule.

Traps

- A changed variable need not move toward a false condition. Specify a bound or decreasing nonnegative quantity.
- In the block-repetition task, `repeating(t,n)` tests whether n is a repetition of a t-digit positive integer. Blocks need their stated width. A last-two-digit block 01 has numeric remainder 1; the source's natural blank fills wrongly accept `repeating(2,101)`. Require the final block to have two digits before comparing blocks.

§3 State a function's job and test its contract

The question. What does omitting an argument mean? How can a test detect an off-by-one loop without testing every input?

Design — CP1.4; Lecture4. Give a function one job, reuse a shared computation instead of copying it, and make the function apply to related inputs. A contract states its domain, return values, and effects. For real inputs, `square` returns a nonnegative real square; this does not assert the same ordered property for complex numbers. **Modularity** divides a program into parts with specific jobs; **abstraction** lets a caller use a contract; **separation of concerns** lets one part change without rewriting another's job.

```
def pressure(v, t, n=6.022e23):
    """Return ideal-gas pressure in pascals; v in m^3, t in K."""
    k = 1.38e-23  # Boltzmann's constant
    return n * k * t / v
```

This is the reading's ideal-gas formula $P = nkT/V$ (cited physical model), with volume in cubic metres, temperature in kelvin, and particle count n. The **default argument** `n=6.022e23` is recorded when the function is defined and used if the call omits that argument. Supplying it overrides the default. A **keyword argument** names the parameter in the call, as in `pressure(v=1,t=273.15)`; positional arguments must come before named ones. The assignment `k = 1.38e-23` instead runs inside each call. A **docstring** documents the function and is shown by `help(pressure)`; a `#` comment is for readers of the source and is ignored during execution.

Example (CP1.4 and Lecture4 slide6)

`pressure(1,273.15)` returns 2269.974834; supplying `3 * 6.022e23` as the third argument returns 6809.924502. For the built-in rounding function: `round(1.23,1)` is 1.2; `round(1.23,0)` is `1.0`; `round(1.23,5)` is 1.23; `round(1.23)` is integer 1. The slide drops the decimal in the second example; Python preserves a floating-point result there.

Assertions and doctests — CP1.5. `assert fib(8) == 21, 'wrong Fibonacci value'` does nothing if the comparison is true; otherwise it raises `AssertionError` with the message. A **unit test** checks one function. Test ordinary inputs and boundaries: $n = 1, 3, 50$ check the Fibonacci loop's zero-iteration case, early progress, and a larger result. A docstring's `>>>` examples specify both displayed output and return values; `python3 -m doctest file.py` checks them. Programmatically, `doctest.testmod()` checks a module's examples; `doctest.run_docstring_examples(f, globals(), True)` checks one function with verbose output. Here `globals()` supplies current global bindings to the examples. The reading tests its loop-based `sum_naturals`, which sums 1 through n, at 10 and 100, expecting 55 and 5050. Passing tests checks those examples; the loop-property argument proves the algorithm for its stated domain.

The reading's `percent_difference(40,50)` computes $100|40 - 50|/40 = 25.0$ (first input must be nonzero). Its absolute-value example uses the negative, zero and positive branches explicitly:

```
def percent_difference(x, y):
    difference = abs(x - y)
    return 100 * difference / x

def absolute_value(x):
    if x < 0:
        return -x
    elif x == 0:
        return 0
    else:
        return x
```

Lecture4's `get_name(first,last)` returns `first` if it is nonempty, otherwise `last`; `get_name('', 'Ousterhout')` gives `'Ousterhout'`. Its `same_length(a,b)` repeatedly divides each positive integer by 10 and compares digit counts: $(4, 8)$ and $(100, 657)$ give true; $(4, 100)$ gives false. That source implementation counts zero as zero digits, so use its positive-integer domain rather than claiming it handles every integer.

Shortcuts

- Distinguish optional arguments from required ones by the signature. For `pressure`, omitting the particle count uses the recorded default; a supplied count overrides it.
- Test a loop that should run zero times, one time, and several times; these cases expose initialization and stopping mistakes.

Traps

- A default is not reassigned on every call. A local assignment and a default parameter use `=` in different execution rules.
- A printed number is not a returned number. Tests that bind `a = divisible_by_k(...)` check the count returned separately from the printed multiples.

§4 Pass the changing operation as a function

The question. The loops for $1 + \cdots + n$, $1^3 + \cdots + n^3$ and a pi approximation differ in just one term. Can one loop express all three?

A **higher-order function** accepts a function as an argument or returns a function. Functions are **first-class values**: they can be named, passed, returned, and stored in collections (the last use is studied later). A parameter can therefore be bound to a function just as it can be bound to a number.

Where the proof comes from

Want. Avoid copying the same summation loop for each formula.

Know. The initial total, index, stopping test and addition stay the same; only the value added at index k changes.

Key step. Supply a one-argument function `term`; use `term(k)` where the differing expression used to be. Bind `term` to the function itself, not to one already-computed term.

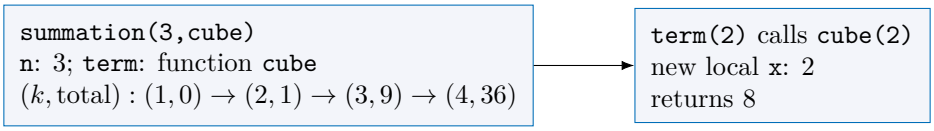
```
def square(x):
    return x * x

def cube(x):
    return x * x * x

def identity(x):
    return x

def pi_term(k):
    return 8 / ((4*k - 3) * (4*k - 1))

def summation(n, term):
    """Sum term(1) through term(n)."""
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total
```



At each index, the same binding `term`→`cube` computes a new value. Before the next test, the total includes exactly the terms already visited.

1. Before the test at index k, `total` equals $\sum_{j=1}^{k-1} \text{term}(j)$. Initially it is the empty sum 0.
2. Adding `term(k)` and increasing k preserves that property. At $k = n + 1$, it gives the complete sum. For $n = 0$, the result is 0 without calling `term`.
3. `summation(100,identity)` is 5050; `summation(100,cube)` is 25502500; `summation(10,square)` is 385. Lecture4's five cubes sum to 225; a separate `cube(7)` call returns 343.

Example (The reading's pi sum)

Its k th term is $8/((4k-3)(4k-1))$. With 100 terms the sum is 3.1365926848388144; with 10^6 terms it is 3.141592153589902. The analytical claim that these sums converge to π is cited from the reading; the code computes the stated finite sums. The original three standalone loops are exactly this common loop with the corresponding term expression substituted.

Change addition into a supplied combination — HW2 Q1–Q2. `from operator import add, mul` supplies two-argument addition and multiplication functions. A product loop starts at 1, because multiplying no factors should leave the multiplicative identity. For a general two-argument `fuse`, start at `start` and repeatedly replace `result` with `fuse(result,term(k))`. This is `accumulate`. HW2 assumes associativity and commutativity; its nested combination can then be evaluated in this left-to-right order. For `accumulate(add,11,3,square)`, totals are $11 \to 12 \to 16 \to 25$. For multiplication with start $= 2$, they are $2 \to 2 \to 8 \to 72$. With zero terms, return the unchanged start and never call either function. Lecture5 defines the repeated-call helper as follows:

```
def apply_twice(f, x):
    return f(f(x))
```

Shortcuts

- Summation uses combination `add` and start 0; product uses `mul` and start 1. The identities explain the two one-line wrappers.
- `apply_twice(square,2)` gives `square`(`square`(2)) = 16: the same function value is used twice, in separate call frames.

Traps

- `summation(5,cube(5))` passes 125, then tries to call it. `summation(5,cube)` passes the required function.
- The name `term` is a local binding, not necessarily the function's original name. Looking up its value uses the same rule as any other parameter.

§5 A returned function retains its defining environment

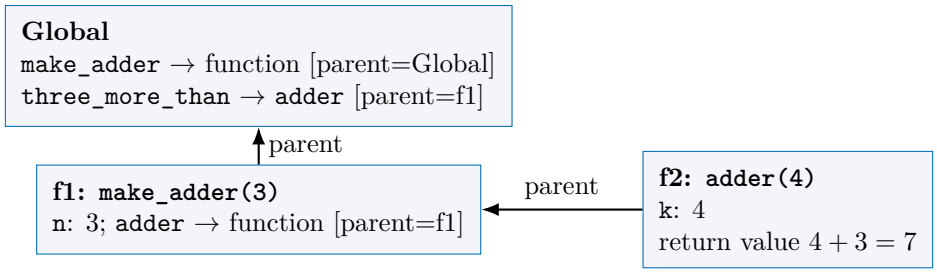
The question. After `three_more_than = make_adder(3)` returns, where does `three_more_than(4)` find the 3?

Proposition (Lexical scope — Lecture5; CP1.6)

When a function is defined, record the current frame as its **parent**. Calling that function creates a local frame with this recorded parent. Name lookup follows those parent links, starting locally. The caller's frame is not substituted for the defining parent. A function retaining access to its enclosing bindings is a **closure**; returning it preserves that access.

```
def make_adder(n):
    def adder(k):
        return k + n
    return adder

three_more_than = make_adder(3)
result = three_more_than(4)
```



Lookup for `n` goes from f2 to f1 and finds 3. The function's recorded parent keeps this binding available after `make_adder` returns.

1. Calling `make_adder(3)` binds `n` in f1. Executing its nested `def` creates `adder` with parent f1; its body is not yet executed.
2. Returning `adder` returns a function value. Calling that value on 4 creates f2 with parent f1, so lookup finds local `k=4`, then enclosing `n=3`.
3. A separate `make_adder(5)` creates another enclosing frame and another function. It does not change the first closure's parent or its 3.

Lecture5's changed line	**Result of `add_three(4)`**
A: `k=5` in the enclosing `make_adder`	7: the inner parameter `k=4` is found first.
B: `n=5` before defining `adder`	9: captured `n` is now 5.
C: `k=5` inside `adder`	8: its local argument binding is replaced.
D: also create `add_five=make_adder(5)`	7: a separate defining frame.
E: `n=5` after nested `def`, before return	9: the closure uses the binding when called.
F: global `n=5` after `make_adder(3)`	7: lookup finds the enclosing 3 first.

Moving `adder` out to global scope changes its parent to Global. If there is no global `n`, calling it raises `NameError`; the caller's local `n=3` cannot supply the missing binding. A closure retains access to bindings, not a rule that copies all free-variable values at definition time (variant E distinguishes these).

Compose and repeat — CP1.6; HW2 Q3–Q4. Composition applies `g` first, then `f`. The returned function's parent holds both function bindings.

```
def compose1(f, g):
    def h(x):
        return f(g(x))
    return h
```

The reading calls the increment function `successor`. `compose1(square,increment)(12)` gives $(12+1)^2 = 169$, even if an unrelated global function is named `f`. To repeat `f` n times, return a function with its own counter and current value: start at input x, replace it by `f(x)` exactly n times, then return it. For three squares starting at 5, the values are $5 \to 25 \to 625 \to 390625$. To test whether `f` and `g` agree in both composition orders at a particular x, return a function computing `f(g(x)) == g(f(x))`; do not claim agreement for *all* inputs from one test.

Lambda creates a function value — CP1.6. A `lambda parameters: expression` has one return expression and no assignment or control statements in its body. Creation does not evaluate that body; a later call does. Its recorded name is `<lambda>`; assigning another name does not rename it. The following assignments rename the reading's `s` and lambda-based `compose1`, keeping both implementations available.

```
square_lambda = lambda x: x * x
compose_lambda = lambda f, g: lambda x: f(g(x))
```

`square_lambda(12)` is 144; `compose_lambda(lambda x:x*x, lambda y:y+1)(12)` is 169. The shorter `compose_lambda` has the same parent-link reasoning as the nested `def`; explicit definitions are often easier to read.

Currying: choose arguments one call at a time — CP1.6. **Currying** transforms a two-argument function into a function returning a one-argument function: `curry2(f)(x)(y)` computes `f(x,y)`. **Uncurrying** reverses that interface.

```
def curried_pow(x):
    def h(y):
        return pow(x, y)
    return h

def curry2(f):
    def g(x):
        def h(y):
            return f(x, y)
        return h
    return g

def uncurry2(g):
    def f(x, y):
        return g(x)(y)
    return f

def map_to_range(start, end, f):
    while start < end:
        print(f(start))
        start = start + 1
```

`curried_pow(2)(3)` is 8. Set `pow_curried=curry2(pow)`: `pow_curried(2)(5)` and `uncurry2(pow_curried)(2,5)` are both 32. `map_to_range(0,10,curried_pow(2))` prints $1, 2, 4, 8, 16, 32, 64, 128, 256, 512$, one per line. The outer call captures 2; each inner call supplies a different exponent. The endpoint 10 is excluded.

A decorator replaces the function binding — CP1.6.

```
def trace(fn):
    def wrapped(x):
        print('-> ', fn, '(', x, ')')
        return fn(x)
    return wrapped

@trace
def triple(x):
    return 3 * x
```

The annotation `@trace` means: create the original `triple`, pass it to `trace`, then bind `triple` to the returned `wrapped` function. Thus `triple(12)` prints a trace line containing the original function and argument 12, and returns 36. The function's displayed memory address varies. This is equivalent to an undecorated definition followed by `triple = trace(triple)`. A decorator may itself be a call expression: evaluate that expression first, create the new function, then apply the resulting decorator to it.

Shortcuts

- For every nested definition, label its parent immediately. Copy that parent to each later call frame; this settles lookup even when the function is called elsewhere.
- Distinguish `return adder` (return the function) from `return adder(4)` (call it and return a number).
- For curried calls, evaluate each call from left to right: `curry2(pow)` returns a function, its `(2)` returns another, and `(5)` returns 32.

Traps

- A global name with the same spelling cannot override an earlier matching binding in the parent chain. Conversely, a caller's unrelated local name is not searched.
- `composite_identity(square,increment)(0)` is true ($1 = 1$), but at 4 it is false ($25 \neq 17$). Its result concerns the supplied input.

§6 Separate an iteration from its update rule

The question. How can one loop approximate both the golden ratio and square roots, when the equations differ?

Supply two functions: an `update` that produces the next guess and a `close` test that decides when the current guess is acceptable. The reading compares numbers by an *absolute tolerance*; $|x - y| < \varepsilon$ is its definition of approximate equality.

```
def approx_eq(x, y, tolerance=1e-15):
    return abs(x - y) < tolerance

def improve(update, close, guess=1):
    while not close(guess):
        guess = update(guess)
    return guess

def golden_update(guess):
    return 1 / guess + 1

def square_close_to_successor(guess):
    return approx_eq(guess * guess, guess + 1)
```

Example (Golden ratio — CP1.6)

The positive number ϕ satisfies $\phi = 1 + 1/\phi$, equivalently $\phi^2 = \phi + 1$. Solving $x^2 - x - 1 = 0$ gives $\phi = (1 + \sqrt{5})/2$. The update $x \mapsto 1 + 1/x$ gives

$$1 \to 2 \to 1.5 \to 1.666666\ldots \to 1.6 \to 1.625 \to \cdots .$$

The residual test compares x^2 with $x + 1$. The printed source implementation returns 1.6180339887498951; comparing this with the closed form at tolerance 10^{-15} passes.

Why this update approaches the positive solution in exact arithmetic.

1. Any positive initial guess maps first to a number greater than 1, then into $(1, 2)$. Subsequent guesses enter and stay in $[1.5, 2]$.
2. For $x, \phi \in [1.5, 2]$, the update error is $|(1 + 1/x) - (1 + 1/\phi)| = |x - \phi|/(x\phi) \leq (4/9)|x - \phi|$.
3. Repeating shrinks the error by a geometric factor. This proves convergence in exact arithmetic; finite-precision stopping still depends on the tolerance and operations.

Capture the equation's parameter — CP1.6 square root. An update `sqrt_update(x,a)` has two inputs, but `improve` requires a one-input update. Put it inside `sqrt(a)` so that it can find `a` through its parent frame.

```
def average(x, y):
    return (x + y) / 2

def sqrt(a):
    def sqrt_update(x):
        return average(x, a / x)
    def sqrt_close(x):
        return approx_eq(x * x, a)
    return improve(sqrt_update, sqrt_close)
```

1. For $a > 0$, a positive guess x and its quotient a/x lie on opposite sides of $\sqrt{a}$ unless already equal. Their average supplies a new positive guess. If $r = \sqrt{a}$, then $(x + a/x)/2 - r = (x - r)^2/(2x) \geq 0$.
2. After the first update, $x \geq r$; the next average is at most x and at least r, the next error is $(x-r)^2/(2x) \leq (x-r)/2$. Repeating halves an upper bound on the error, so the exact guesses converge to r.
3. For `sqrt(256)`, the local update call looks up `x` in its own frame and `a=256` in the enclosing `sqrt` frame; it does not look up `a` in `improve`'s frame. The source example returns `16.0`.

The **derivative** $df(x)$ is the slope of the tangent line approximating the curve near x.

Proposition (Newton update — CP1.6)

Given a differentiable function f and its derivative df, the tangent line at x is $y = f(x) + df(x)(t - x)$. If $df(x) \neq 0$, its zero occurs at

$$\boxed{x_{\text{new}} = x - \frac{f(x)}{df(x)}.}$$

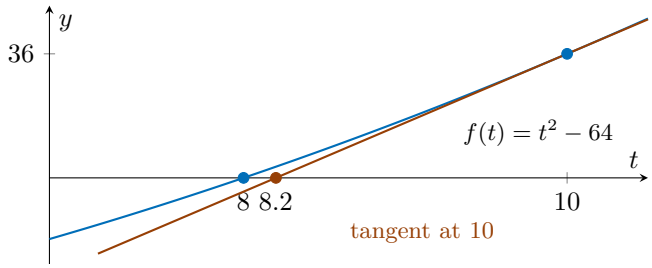
This is an update rule, not a guarantee of convergence for arbitrary functions or initial guesses.

Where the proof comes from

Want. Move a guess toward a zero of a curved function.

Know. Near the guess, the tangent is a linear approximation. Solving the exact curved equation may be difficult; solving a linear equation is direct.

Key step. Set the tangent's output to zero and solve for its input: $0 = f(x) + df(x)(t - x)$. This suggests a new guess; check the original function again there.



At $x = 10$, $f(x) = 36$ and $df(x) = 20$. The tangent meets zero at $10 - 36/20 = 8.2$, closer to the curve's actual zero 8.

```
def newton_update(f, df):
    def update(x):
        return x - f(x) / df(x)
    return update

def find_zero(f, df):
    def near_zero(x):
        return approx_eq(f(x), 0)
    return improve(newton_update(f, df), near_zero)
```

Roots of different degrees — CP1.6. For positive integer n and $a > 0$, the equation $x^n = a$ becomes the zero equation $f(x) = x^n - a = 0$. The calculus rule $df(x) = nx^{n-1}$ is cited. For nonnegative integer n, the power loop multiplies by x exactly n times; it returns 1 for $n = 0$.

```
def square_root_newton(a):
    def f(x):
        return x * x - a
    def df(x):
        return 2 * x
    return find_zero(f, df)

def power(x, n):
    product, k = 1, 0
    while k < n:
        product, k = product * x, k + 1
    return product

def nth_root_of_a(n, a):
    def f(x):
        return power(x, n) - a
    def df(x):
        return n * power(x, n - 1)
    return find_zero(f, df)
```

At $a = 64$, degrees $2, 3, 6$ give $8.0, 4.0, 2.0$: $8^2 = 4^3 = 2^6 = 64$. `square_root_newton(64)` also returns 8.0.

Shortcuts

- To derive a Newton update, first write the desired equation as $f(x) = 0$, then solve the tangent equation. For $x^2 - a$, this gives $x - (x^2 - a)/(2x) = (x + a/x)/2$, the same average update.
- Nested helpers let a general one-input method use a fixed parameter: its value is stored in the enclosing frame, not added as another update argument.

Traps

- Near a zero, residual $|f(x)|$ and input error $|x - r|$ at a zero r are different quantities. For example, $f(x) = 10^{-6}x$ at $x = 1$ has residual 10^{-6} but input error 1. A residual tolerance alone is not a universal bound on input error.
- Smaller tolerance need not make floating-point iteration finish. For $a = 10$, the average update can become unchanged at 3.162277660168379, whose squared residual is about $-1.77636 \cdot 10^{-15}$: the source's 10^{-15} test keeps failing. Its iframe diagrams instead use 10^{-3}; do not confuse those two tolerances.
- Newton can divide by zero, diverge, or alternate forever: for $f(x) = x^3 - 2x + 2$, initial guess 0 updates to 1, then back to 0. A test on the successful 64 examples does not prove general convergence.

§7 Use function contracts to separate a game from its decisions

The question. Can a simulator play the same game with different players, scoring rules and dice, without rewriting its turn loop?

Lecture4's Twenty-One. Start at total = 0; alternate adding 1, 2, or 3. The first player to make the total at least 21 *loses*. A strategy takes the current total and returns an allowed addition. The source's `simple_strategy` adds 3 below 15, then 2; its nested wrapper prints a supplied remark before calling the original strategy. The simulator below is the lecture's `play`, renamed to distinguish it from Hog.

```
def simple_strategy(score):
    if score < 15:
        return 3
    return 2

def make_heckling_strategy(strategy, heckle):
    def heckling_strategy(s):
        print(heckle)
        return strategy(s)
    return heckling_strategy

def play_twenty_one(strategy0, strategy1):
    current_score, current_player = 0, 0
    while current_score < 21:
        if current_player == 0:
            current_score += strategy0(current_score)
        else:
            current_score += strategy1(current_score)
        current_player = 1 - current_player
    print('Player', current_player, 'wins the game')
```

With two simple strategies, totals are $0, 3, 6, 9, 12, 15, 17, 19, 21$. Player1 makes the last addition and loses; the code has already switched `current_player` to Player0, so it prints `Player 0 wins the game`. The lecture's `interactive_strategy` prints the current score, reads text with `input()`, and converts it using `int`; only the strategy changes, not the simulator.

§7.1 Hog: define a die, a turn, and a score update

The question. If a die rolls a 1 on the second of five required rolls, can the simulator stop rolling immediately?

P0: die interface. A die is a supplied zero-argument function returning a positive integer outcome. A fair six-sided die returns 1–6 with equal probabilities. The supplied `make_test_dice(4,1,2)` returns a stateful test die whose successive calls return $4, 1, 2, 4, 1, \ldots$. Its implementation uses later features; its documented interface is enough here. Each call advances its state, so repeated calls are observable effects.

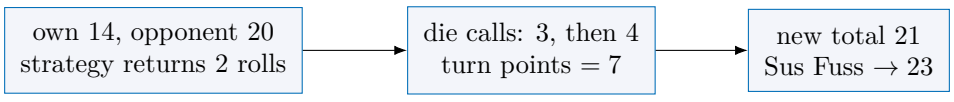
Rules. Two players alternate turns toward a goal (default 100). A strategy takes the current player's score and the opponent's score and returns a roll count 0–10. For positive rolls, **Sow Sad** awards 1 if any roll is 1; otherwise award the sum. For zero rolls, **Boar Brawl** awards

$$\max\big(1, 3|\text{opponent's tens digit} - \text{own ones digit}|\big).$$

Then **Sus Fuss** applies to the *new total*: if it has exactly 3 or 4 positive factors, replace it with the next greater prime. A **prime** is an integer at least 2 with exactly the positive divisors 1 and itself. Both 1 and the number count as factors. The game ends at the end of a turn when a total reaches or exceeds the goal.

Example (Actual Hog examples)

Seven dice with five 1 s score 1; four 3 s score 12. Zero-roll scores for own/opponent totals $(21, 46)$, $(45, 52)$ and $(2, 5)$ are $9, 1, 6$. A total $14 + 7 = 21$ has factors $1, 3, 7, 21$, so it becomes 23; $63 + 1 = 64$ has seven factors and stays 64; $49 + 18 = 67$ is prime and stays 67.



A strategy chooses a roll count; a turn produces points; an update returns a new total. Confusing these three return values causes different errors.

Derive each simulator function from its contract — P1–P5.

1. `roll_dice` records a sum and whether any 1 occurred. Make *every* requested die call, then return 1 or the sum. Early return on the first 1 would leave a test die at the wrong point in its cycle.
2. `boar_brawl` extracts `score % 10` and `(opponent // 10) % 10`; the latter must retain only the tens digit even above 99. `take_turn` dispatches zero rolls here and positive rolls to `roll_dice`.
3. `num_factors` counts divisors 1 through n. `sus_points` advances from a sus total until the supplied positive-integer `is_prime` test passes. `sus_update` adds turn points to the old total *before* applying Sus Fuss. The next-prime search terminates because primes exist above every integer (cited number-theory fact).
4. `play` stores both scores and whose turn is next. On each turn, call only that player's strategy once, pass its result to the update once, replace only that player's total, and switch players. For Player1, reverse the score arguments so its own score is still first. Return the ordered pair `score0, score1`.

Each turn adds at least one point under these rules, so a game with a positive goal terminates. If either initial score already reaches the goal, no turn is needed. A comma-separated return of both scores forms an ordered pair; the caller can unpack it into two names, as with Week1 simultaneous assignment.

Returned strategies and finite checking — P6–P7. `always_roll(n)` returns a two-score strategy retaining `n` in its parent frame. To check a deterministic strategy for a goal = 100 game, compare its result at every pair (s, o) with $0 \le s, o < 100$: $100^2 = 10000$ pairs. Two nested `while` loops enumerate them. One differing result proves it is not constant. This finite certificate assumes a deterministic strategy; it cannot certify all future behavior of an arbitrary stateful function.

Average repeated calls with the same arguments — P8–P9. The new syntax `def averaged(*args)` accepts an arbitrary number of positional arguments as an ordered group; `f(*args)` passes those same arguments to `f`. It does not call `f` when the wrapper is defined. Hog illustrates it with a printing wrapper:

```
def printed(f):
    def print_and_return(*args):
        result = f(*args)
        print('Result:', result)
        return result
    return print_and_return
```

`printed(pow)(2,8)` prints `Result: 256` and returns 256; `printed(abs)(-10)` prints `Result: 10` and returns 10. An averaging wrapper instead calls `f(*args)` a fixed positive number m of times and returns $(r_1 + \cdots + r_m)/m$. With test outcomes $4, 2, 5, 1$ and 40 one-die samples, the mean is $10(4 + 2 + 5 + 1)/40 = 3.0$. The same die is used throughout; do not recreate it inside each sample.

`max_scoring_num_rolls` evaluates counts $1, 2, \ldots, 10$ in that order. Record the largest sampled mean and update the best count only on a *strict* improvement, preserving the smaller count on ties. With a constant die 3, ten rolls are best; with constant 1, every mean is 1, so choose one roll. Random sample means estimate performance; they do not prove a globally optimal strategy.

Compare a zero-roll gain with a threshold — P10–P12. `boar_strategy` ignores Sus Fuss and compares Boar Brawl points with the threshold. `sus_strategy` instead computes `sus_update(0,score,opponent)-score`. For $(53, 60)$, zero rolls give 9 points, new total = 62, Sus Fuss raises it to 67, and the gain is 14. Return zero at threshold 14, but the fallback at threshold 15. Optional `final_strategy` can first take a certain zero-roll win; this is one concrete improvement, not a claim of optimality.

Win-rate estimates and printing wrappers.

1. The source's `winner` returns 0 if Player0's final score is greater, and 1 otherwise. A Player0 win therefore has indicator $1 - \text{winner}(\text{strategy}, \text{baseline})$; a Player1 win has indicator $\text{winner}(\text{baseline}, \text{strategy})$.
2. `average_win_rate` averages repeated indicators in each position, then averages those two fractions. The result estimates performance against the supplied baseline; random estimates vary by run.

The actual wrappers in `hog_ui.py` preserve those same game interfaces:

- `printing_strategy(0,always_roll_5)` returns a strategy that, at scores $(10, 20)$, prints the scores and its choice of 5 dice, then returns 5.
- `printing_dice` returns a zero-argument die that prints each outcome and returns it. `sus_update_and_print` prints the turn calculation and returns the updated total: outcomes $4, 5, 3$ at own score 9 give $9 + 12 = 21 \to 23$.

- `play_and_print` passes the wrappers to the unchanged simulator. Running `python3 hog_ui.py -n 1` supplies one human-input strategy, `-n 2` two. The supplied `hog_gui.py` instead provides a graphical interface with the same scoring rules.

Shortcuts

- Write each callback's inputs and return value beside its parameter before composing it. `update` returns a total; adding the old score again double-counts it.
- Test with a cyclic die and inspect the next outcome afterwards; this catches too few or too many calls even when a turn's score looks correct.
- Phase1 checkpoint is September10; full project September17. Use the provided tests before the UI; a UI displays the same simulator bugs.

Traps

- A zero-roll turn calls no die. Boar Brawl's award is at least 1, and Sus Fuss tests the resulting total, not the award alone.
- `is_always_roll` must test off-diagonal score pairs as well as equal scores; `catch_up` returns 6 when behind and 5 otherwise.
- Do not reset `make_averaged`'s captured die or its iteration count while computing a sample. The returned function must accept the *original function's* arguments.

§8 Represent numbers by function iteration (HW2 optional)

The question. How can a function represent the number 3 without storing the integer 3 in its body?

A **Church numeral** N representing $n \geq 0$ takes a function f and returns a function that applies f exactly n times to an input. Write $f^{[n]}(x)$ for that repeated application; this notation denotes composition, not multiplication of return values. Thus $N(f)(x) = f^{[n]}(x)$; $f^{[0]}(x) = x$.

1. Zero returns the identity function; one returns a function applying f once; two applies it twice. A successor adds one outer f: $f(N(f)(x))$.
2. To recover the ordinary integer, take $f(x) = x + 1$ and start at 0. Applying it n times gives n.
3. To add $m + n$, apply f n times and then m more: $M(f)(N(f)(x))$.
4. To multiply mn, use $N(f)$ as the operation repeated m times: $M(N(f))(x)$. Each of those m operations applies f n times.
5. For m^n, repeat the numeral M as an operation on functions n times: $N(M)$. For $m = 2, n = 3$, its action on f produces $f \to f^{[2]} \to f^{[4]} \to f^{[8]}$. Hence it is the numeral for 8.

These are the source's just-for-fun puzzles, explicitly outside the required course scope. Full code and worked source tests are in the separate solutions.

Shortcuts

- Check a proposed numeral by applying it to increment and then 0, and also by applying it to another function. The contract concerns repeated application, not only one conversion result.

Traps

- $M(N(f))$ multiplies application counts; $M(f)(N(f)(x))$ adds them. Distinguish the function supplied as an argument from the input supplied later.
- The source requires `one` and `two` directly; calling `successor` to implement them violates that restriction.

§9 Problem set

Lab1 Q1: Return and Print. Predict the full prompt display for `welcome()`, then for `print(welcome(),cal())`.

```
def welcome():
    print('Go')
    return 'hello'

def cal():
    print('Bears')
    return 'world'
```

Hint: Evaluate the two operands left to right; record each printed line separately from the strings returned.

Lab1 Q2: WWPD: What If? Predict the full prompt display for `ab(10,20)`, `bake(0,5)`, `bake(1,'yum')`, and `bake(2,3)`.

```
def ab(c, d):
    if c > 5:
        print(c)
    elif c > 7:
        print(d)
    print('foo')

def bake(cake, make):
    if cake == 0:
        cake = cake + 1
        print(cake)
    if cake == 1:
        print(make)
    else:
        return cake
    return make
```

Hint: The two `if`s in `bake` are separate; the `else` belongs to the second. Printing does not exit a call.

Lab1 Q3: Pick a Digit. For positive integer n and nonnegative integer k, return the digit k places left of the rightmost digit, or 0 if absent. Use a single return statement. Tests: $(3579, 2) \mapsto 5$, $(3579, 0) \mapsto 9$, $(3579, 10) \mapsto 0$.

Hint: Remove k rightmost digits using division by 10^k, then extract one digit with remainder.

Lab1 Q4: Middle Number. Return the middle of three distinct integers using one return expression. Tests $(3, 5, 4) \mapsto 4$, $(30, 5, 4) \mapsto 5$, $(3, 5, 40) \mapsto 5$, $(30, 5, 40) \mapsto 30$.

Hint: Sum all three, then remove the minimum and maximum; arithmetic is exact for these integers.

Lab1 Q5: Falling Factorial. Return $n(n-1)\cdots(n-k+1)$, where k is a nonnegative integer counting the consecutive descending factors; for $k = 0$, return 1. Tests $(6, 3) \mapsto 120$, $(4, 3) \mapsto 24$, $(4, 1) \mapsto 4$, $(4, 0) \mapsto 1$.

Hint: Keep the product of the factors already used and the number of factors remaining.

Lab1 Q6: Divisible By k. For positive integers n, k, print all positive multiples of k at most n, in increasing order, and return their count. Test $(10, 2)$: print $2, 4, 6, 8, 10$, return 5; $(3, 1)$ returns 3; $(6, 7)$ prints nothing and returns 0.

Hint: Start at k and advance by k; a separate counter records printed lines.

Lab1 Q7: Double Eights. For a nonnegative integer, determine whether the decimal digits contain adjacent 8 s. Source true cases: $88, 2882, 880088$; false: $8, 12345, 80808080, 1077$.

Hint: Check the last two digits, then remove only the last digit.

Lab1 Q8: Syllabus Quiz. The official Syllabus Quiz is a course-policy form that requests sign-in. Its question text is unavailable in the public materials.

Hint: Use the actual linked form and official syllabus when reviewing that administrative requirement.

Lab1 Q9 (optional): Sum Digits. Sum the decimal digits of a nonnegative integer. Tests $10 \mapsto 1$, $4224 \mapsto 12$, $1234567890 \mapsto 45$, $123 \mapsto 6$.

Hint: Add the last digit and remove it; before each test, the accumulator is the sum already removed.

Discussion1 Q1: Race. The tortoise walks x feet/minute. The hare alternates five minutes at y feet/minute and five resting, with integers $0 < x < y \leq 2x$. Find an input for which the supplied function is wrong or runs forever, and explain why. The source's intended equality examples are `race(5,7)`$= 7$ and `race(2,4)`$= 10$.

```
def race(x, y):
    assert y > x and y <= 2 * x
    tortoise, hare, minutes = 0, 0, 0
    while (minutes == 0) or (tortoise - hare):
        tortoise += x
        if minutes % 10 < 5:
            hare += y
        minutes += 1
    return minutes
```

Hint: A nonzero signed distance difference is true on either side of zero. A first crossing may skip equality.

Discussion1 Q2: Fizzbuzz. For each integer 1 through positive n, print `fizzbuzz` for multiples of both 3 and 5, `fizz` for other multiples of 3, `buzz` for other multiples of 5, otherwise the number. The source tests $n = 16$ and confirms the returned value is `None`.

Hint: Put the joint condition first in an `if/elif/else` chain.

Discussion1 Q3: Is Prime? Return whether positive integer n is prime. Tests: $10 \mapsto$ `False`, $7 \mapsto$ `True`, $1 \mapsto$ `False`. Describe the process in one sentence.

Hint: Handle $n < 2$, then search for a divisor from 2 through $n - 1$; one divisor suffices to disprove primality.

Discussion1 Q4: Unique Digits. Implement `has_digit(n,k)` for digit $0 \leq k < 10$, and call it from `unique_digits(n)` to count the distinct digits of positive n. Tests: $8675309 \mapsto 7$, $13173131 \mapsto 3$, $101 \mapsto 2$; `has_digit(10,1)` true, `has_digit(12,7)` false.

Hint: Check each of the ten possible digits once; each helper searches the original number independently.

Discussion1 Q5: Bottles. For the code below, explain what determines the number of call frames, what use is made of `pass_it`'s return value, and whether its local assignment changes the global frame.

```
bottles = 99
take = 1

def pass_it(around):
    bottles = 98
    return take

remaining = bottles - pass_it(bottles)
bottles = remaining
```

Hint: Count executed user-function calls, then distinguish global `bottles` from the callee's local name.

Discussion1 Q6: Double Trouble. Draw the environment diagram after this code. Which names refer to the same function? What do `hat(4)` and `double(4)` return?

```
def double(x):
    return x * 2

def triple(x):
    return x * 3

hat = double
double = triple
```

Hint: Assignment copies the current function value, not a reference to the other variable name. The source asks for the diagram; the two calls are additional checks.

Discussion1 Q7 (optional; Spring2023 MT1 variant): Repeating. Return whether positive integer n consists of one or more repeats of some t-digit positive integer. For $n = 6161$, results for $t = 1, 2, 3, 4, 5$ are false, true, false, true, false. No `for`, `range`, lists, or string conversion.

Hint: Save the last t digits and compare equal-width blocks while removing them. Also require that saved block to have t digits: the worksheet's original initial-length guard alone misses this case.

HW2 Q1: Product. For positive integer n and one-argument `term`, return `term(1)*...*term(n)`. Supplied helpers: identity, square, increment, triple. Source tests: product of first 3 identities = 6, first 5 identities = 120, first 3 squares = 36, first 5 squares = 14400, first 3 increments = 24, first 3 triples = 162.

Hint: Use the summation loop's maintained property with multiplication and initial result 1.

HW2 Q2: Accumulate. Combine `start` and the first $n \geq 0$ terms using a supplied two-argument `fuse`. Assume it is commutative ($f(a, b) = f(b, a)$) and associative ($f(f(a, b), c) = f(a, f(b, c))$). Then implement `summation_using_accumulate` and `product_using_accumulate`, each as a single return line calling `accumulate`.

Hint: The start value is returned unchanged for zero terms; choose identity starts 0 and 1 for the wrappers.

HW2 Q3: Make Repeater. Return a function applying the supplied one-argument `f` exactly positive integer n times. Tests: three increments at $5 \mapsto 8$; five triples at $1 \mapsto 243$; two squares at $5 \mapsto 625$; three squares at $5 \mapsto 390625$.

Hint: Capture `f,n` in the defining frame; create the current value and counter separately for each returned-function call.

HW2 Q4: Composite Identity Function. Return a one-argument function testing `f(g(x)) == g(f(x))`; each composition's intermediate value is a valid input for the other function. Square and increment agree at 0, but not at 4.

Hint: Return the comparison function; do not perform a comparison before its input x is supplied.

HW2 Q5 (just for fun, outside required course scope): Church Numerals. Given `zero` and `successor`, implement `one` and `two` directly without calling `successor`; then implement conversion `church_to_int`, addition `add_church`, multiplication `mul_church`, and power `pow_church`. Source checks convert zero/one/two/three to 0/1/2/3, $2 + 3$ to 5, $2 \cdot 3$ to 6, $3 \cdot 4$ to 12, 2^3 to 8, 3^2 to 9.

Hint: Reason about how many applications of an arbitrary `f` each returned function performs.

§9.1 Hog project: Phase1 due September10, complete project September17

Hog P0: Dice. Predict five successive calls to `make_test_dice(4,1,2)`'s returned die. Explain why it is not pure.

Hint: The supplied die retains its position in a repeating cycle.

Hog P1: Roll Dice. Implement `roll_dice(num_rolls,dice=six_sided)` for positive roll count, with Sow Sad. Call the die exactly `num_rolls` times, including after a 1. No lists, strings, square brackets, or `for` statements; preserve the supplied assertions.

Hint: Keep both a sum and a Boolean recording whether any 1 has occurred; choose the return value only after all calls.

Hog P2: Boar Brawl. Return the zero-roll award from own and opponent scores; handle arbitrary nonnegative scores, including above 99. No lists, strings, square brackets, or `for` statements.

Hint: Extract own ones and opponent tens with remainder; award at least 1.

Hog P3: Take Turn. Implement `take_turn` for roll counts 0–10, calling `boar_brawl` or `roll_dice` instead of repeating their implementations; preserve supplied assertions.

Hint: Dispatch on zero; pass the supplied die through only to the positive-roll case.

Hog P4: Sus Fuss. Implement `num_factors`, `sus_points`, and `sus_update`. Use the supplied `is_prime` and the earlier functions. `sus_points` returns an updated total, unchanged unless it has 3 or 4 factors.

Hint: Count inclusive divisors; find the next prime only for a sus total; apply the rule after adding the turn's points.

Hog P5: Play. Simulate alternating turns with supplied strategies and update, optional starting scores/die/goal. Call only the current player's strategy, once per turn; return final totals in Player0, Player1 order.

Hint: Store whose turn is next and reverse the score arguments for Player1. An update returns a total, not a gain.

Hog P6: Always Roll. Return a strategy that accepts two scores and always returns the captured $0 \leq n \leq 10$.

Hint: The outer function chooses n; the returned function accepts scores later.

Hog P7: Is Always Roll. Check whether a deterministic strategy returns the same roll count for every pair of scores below the specified positive goal.

Hint: Two nested loops cover every pair, including unequal scores; stop at the first mismatch.

Hog P8: Make Averaged. Return a function accepting the same arguments as the supplied function, calling it `iterations` times on those arguments, and returning the mean. Default iterations = 1000; use `*args`.

Hint: Capture the function and positive iteration count; create fresh sum and counter on each returned-function call.

Hog P9: Max Scoring Num Rolls. Use `make_averaged` and `roll_dice` to compare roll counts 1–10 in ascending order, returning the smaller count on equal sampled means.

Hint: Replace the current best only on a strict improvement.

Hog P10: Boar Strategy. Return zero rolls if Boar Brawl gives at least the threshold, otherwise `num_rolls`. Ignore Sus Fuss. Defaults: threshold = 11, fallback = 6.

Hint: Compare the direct zero-roll award, not a new total.

Hog P11: Sus Strategy. Same interface/defaults as P10, but compare the gain after both zero-roll and Sus Fuss rules with the threshold.

Hint: Subtract the starting score from `sus_update(0,score,opponent)`.

Hog P12 (optional): Final Strategy. Supply a valid strategy using the rules to improve play against the baseline. There is no unique required algorithm.

Hint: A zero-roll turn that certainly reaches the goal already wins; otherwise choose a justified baseline rule and test it.

§9.2 Actual past-exam practice

Fall2022 Midterm1 Q1(c), reproduced in Lecture5 slide4. Predict every printed line of `print(print(print(s,s) or print("Who's There?")), "Who?")` with `s='Knock'`.

Hint: Each inner print returns false-valued `None`; `or` therefore evaluates its right operand.

Fall2022 Midterm1 Q3(a), reproduced in Lecture5 slides6–7. For integer $n > 1$, sum its proper factors and return `'deficient'`, `'perfect'`, or `'abundant'` when that sum is respectively below, equal to, or above n. Tests: 6 perfect, 24 abundant, 23 deficient.

Hint: Sum divisors strictly below n, then compare the resulting total with n.

Fall2026 Midterm1 Q3: Long Odds, p.4. A group is a maximal consecutive sequence of all-odd or all-even decimal digits. Count the groups of positive n with length at least positive k. For 15732429999, the groups are $1573, 242, 9999$; at thresholds $3, 4, 5, 1$ the counts are $3, 2, 0, 3$.

The supplied loop increases `group_size` before each digit removal, increments `count` at condition(a), resets `group_size=0` at condition(b), then sets `n=n//10`. Choose(a) from `group_size < k`, `<= k`, `> k`, `>= k`, `== k`, `!= k`, `% k == 0`, `% k != 0`; fill(b). In the two answers, do not write `str`, `len`, `if`, `else`, square brackets, or digits 3–9.

Hint: Count a group exactly when it first reaches the threshold; a parity change separates groups. The given loop already contains its two `if` statements.

```
def num_long_groups(n, k):
    assert n > 0 and k > 0, 'n and k must be positive.'
    group_size, count = 0, 0
    while n:
        group_size += 1
        if _____:  # (a)
            count += 1
        if _____:  # (b)
            group_size = 0
        n = n // 10
    return count
```

Fall2026 Midterm1 Q4.1–Q4.2: Space Between, p.5. For $n > 1$, implement `gap(n,term)`, the largest absolute difference between consecutive values `term(1),...,term(n)`. In the supplied loop beginning `k,largest=1,0`, choose the condition from `k < n-1`, `k < n`, `k < n+1`, `k < n+2`, and fill the updated `largest`. Then implement `odd_gap(n,term)` for $n > 2$ by filling `return gap((n+1)//2, _____)` so it compares only indices $1, 3, 5, \ldots \leq n$. Source checks: `gap(5,square)`$= 9$, `gap(6,lambda x:x%4)`$= 3$, `gap(4,lambda x:-x)`$= 1$, `odd_gap(5,square)`$= 16$, `odd_gap(8,lambda x:x%5)`$= 3$. No `if`, `else`, square brackets, `for`, or `range` in the answers.

Hint: Visit adjacent index pairs $(k, k+1)$; for odd indices, translate index j to $2j - 1$ inside a function.

```
def gap(n, term):
    k, largest = 1, 0
    while _____:  # (a)
        largest = _____  # (b)
        k = k + 1
    return largest

def odd_gap(n, term):
    return gap((n + 1) // 2, _____)  # (c)
```

Attribution. Adapted examples from John DeNero and contributors, *Composing Programs*, third edition, sections1.4–1.6, CC BY-NC-SA4.0; official Fall2026 CS61A lectures/assignments. Source identities, exact examples and corrections are recorded in `audit.md`.